Pion Soft Factor Measurement

A Detailed Physics Note for the PyQUDA Implementation

May 19, 2026

1 Physics Overview

The pion soft factor is a four-point correlation function that appears ubiquitously in the factorization of transverse-momentum-dependent (qTMD) observables. In the Collins–Soper (CS) framework [1, 2] and the Ji–Ma–Yuan (JMY) framework [3, 4], the soft factor absorbs rapidity divergences that arise when one attempts to separate the hard scattering, the collinear beam functions, and the soft radiation in hadron scattering processes.

For a pion, the soft factor is defined through a matrix element of four Wilson-line staple operators arranged on the light-cone. In a Euclidean lattice calculation one gives up the light-cone separation and instead works with equal-time correlators in a large-momentum frame, following the LaMET/quasi-PDF philosophy. The Lorentz invariant definition is then replaced by a quasi-soft-factor whose infrared structure coincides with the physical soft factor after a perturbative matching.

Concretely, the lattice object we compute is

$$S(b_\perp, t; t_0, t_{\text{sep}}, \Gamma_1, \Gamma_2) \;=\; \sum_{\mathbf{x}} \mathrm{Tr}\left[A_b(\mathbf{x}, t)\, \Gamma_2\, B_b(\mathbf{x}, t)\, \Gamma_1 \right], \tag{1}$$

where $b_\perp$ is a transverse displacement between the quark and antiquark Wilson lines, t_0 is the source time slice, t_{sep} is the sink–source separation, and the closed spin-color blocks A_b and B_b are built from four Coulomb-gauge-fixed wall-source propagators as described in detail below. The matrices Γ_1 and Γ_2 are drawn from a reduced gamma set appropriate for the soft factor.

The physical interpretation of Eq. (1) is that the two blocks A_b and B_b represent the quark and antiquark lines that travel from the source to the sink, with the transverse displacement b inserted to probe the transverse-momentum dependence of the soft radiation. The trace over spin and color indices projects onto the desired gamma structures.

2 Two-Stage Workflow

The calculation is intentionally split into two independent stages to allow flexible reuse of the expensive propagator inversions.

Stage 1 – Propagator Generation (`Pyquda_pion_soft_factor_prop.py`)

- Read a Coulomb-gauge-fixed configuration.
- For every requested source time slice t_0 and every quark momentum $\mathbf{k}$ (and $-\mathbf{k}$), construct a momentum-phase wall source and invert the Dirac operator.
- Save each propagator as an HDF5 file with full metadata (lattice tag, configuration number, smearing tag, source position, quark momentum).

Stage 2 – Contraction (`Pyquda_pion_soft_factor_contract.py`)

- Load the wall propagators saved in Stage 1 (no gauge field needed except for lattice geometry metadata).
- For each pair of forward/backward quark momenta $(\mathbf{k}_{\rm fw}, \mathbf{k}_{\rm bw})$ compute:
  (i) the wall-to-wall two-point diagnostic $C_2(t)$,
  (ii) the qTMDWF-like diagnostic $C_{\rm TMDWF}(b_T, b_z, t)$,
  (iii) the full soft-factor four-point correlator $S(b_\perp, t; t_{\rm sep}, \Gamma_1, \Gamma_2)$.
- Save all correlators to HDF5 with appropriate group hierarchies.

This two-stage design means that one can change contraction parameters (e.g. Γ_1, Γ_2, b_T ranges, sink interpolators) without re-running any inversions.

3 Wall-Source Propagators

3.1 Coulomb-Gauge Wall Source

The source for quark momentum $\mathbf{k}$ at time t_0 is

$$\eta_{\mathbf{k}}(\mathbf{x}, t) \;=\; \delta_{t,\,t_0}\; \exp\big(+\,i\,\mathbf{k}\cdot\mathbf{x}\big)\,. \tag{2}$$

The spatial part is a plane wave with momentum $+\mathbf{k}$, applied uniformly on every spatial site of time slice t_0. The Coulomb gauge condition is imposed before the wall source is constructed, which suppresses the gauge noise that would otherwise contaminate a point source. The corresponding propagator is

$$G_{\mathbf{k}}(x; t_0) \;=\; D^{-1}\,\eta_{\mathbf{k}}\,, \tag{3}$$

where D is the Wilson-clover Dirac operator with appropriate boundary conditions (t-boundary set to antiperiodic, `t_boundary = -1`).

3.2 Both Signs of Quark Momentum

The soft-factor contraction requires both $+\mathbf{k}$ and $-\mathbf{k}$ propagators on all time slices. Stage 1 therefore loops over a set of momenta that includes both signs:

$$\mathcal{K}_{\rm save} \;=\; \{\,\mathbf{k}_{\rm fw},\, \mathbf{k}_{\rm bw},\, -\mathbf{k}_{\rm fw},\, -\mathbf{k}_{\rm bw}\,\} \quad \text{(duplicates removed)}. \tag{4}$$

3.3 Antiquark Line from Gamma5 Hermiticity

The Euclidean Dirac operator satisfies γ_5-hermiticity,

$$D^\dagger \;=\; \gamma_5\, D\, \gamma_5\,. \tag{5}$$

For an antiquark propagating *backward* in time the required propagator is obtained from the $-\mathbf{k}$ forward propagator via

$$\bar{G}_{-\mathbf{k}}(x; t_0) \;=\; \gamma_5\, G_{-\mathbf{k}}(x; t_0)^\dagger\, \gamma_5\,. \tag{6}$$

In the code this operation is performed by the `einsum` pattern

```
xp.einsum("ij, tzyxmlca, kl -> tzyxkjca",
          gamma5, prop.conj(), gamma5)
```

which transposes the spin indices ($i \leftrightarrow j,\ k \leftrightarrow l$) appropriately. The color index a and the spatial lattice indices (z, y, x) are spectators in this transformation.

3.4 Momentum Phase Convention

The phase convention is tied to the definition of the wall source, Eq. (2):

Operation	Phase factor
Wall source at t_0 with momentum $\mathbf{k}$	$\exp(+i\,\mathbf{k}\cdot\mathbf{x})$
Sink-side phase on $G_{\mathbf{k}}$ (two-point)	$\exp(-i\,\mathbf{p}\cdot\mathbf{x})$ where $\mathbf{p}$ is the meson momentum
Sink-side phase on $G_{\mathbf{k}_{\rm bw}}$ (soft factor)	$\exp(-2i\,\mathbf{P}\cdot\mathbf{x})$ where $\mathbf{P}=\mathbf{k}_{\rm fw}+\mathbf{k}_{\rm bw}$

The sign of the applied phase in `apply_phase` is controlled by the `sign` argument. When `sign=1`, the raw momentum-phase factor $\exp(+i\mathbf{k}\cdot\mathbf{x})$ is multiplied onto the propagator data. When `sign=-1`, the complex conjugate $\exp(-i\mathbf{k}\cdot\mathbf{x})$ is used instead.

4 Wall-to-Wall Two-Point Diagnostic

Before computing the four-point soft factor we verify that the wall-source propagators produce a sensible pion two-point function. Using the forward propagator $G_{\rm fw}\equiv G_{\mathbf{k}}(t_0)$ and the backward-source propagator $G_{\rm bw}\equiv G_{-\mathbf{k}_{\rm bw}}(t_0)$, the wall-to-wall two-point function for source interpolator $\Gamma_{\rm src}$ and sink interpolator $\Gamma_{\rm sink}$ is

$$C_2(t) \;=\; \sum_{\mathbf{x}} \mathrm{Tr}\left[\Gamma_{\rm src}\,\bar{G}_{\rm bw}(\mathbf{x},t;t_0)\,\Gamma_{\rm sink}\,G_{\rm fw}^{\rm phased}(\mathbf{x},t;t_0)\right], \tag{7}$$

where

- $\bar{G}_{\rm bw}$ is the gamma5-hermiticity-conjugated antiquark line, Eq. (6).
- $G_{\rm fw}^{\rm phased}$ is the forward propagator multiplied by the meson momentum phase $\exp(-i\mathbf{p}\cdot\mathbf{x})$ with $\mathbf{p}=\mathbf{k}_{\rm fw}+\mathbf{k}_{\rm bw}$.
- The backward propagator is *not* phased, because the wall source $\eta_{-\mathbf{k}_{\rm bw}}(t_0)$ already carries $-\mathbf{k}_{\rm bw}$ in its construction; the antiquark line thus naturally projects onto total momentum $\mathbf{p}$.

In the code, `contract_wall_2pt` performs this computation using three nested `einsum` calls:

1. $\bar{G}_{\rm bw}$ via gamma5 conjugation.
2. $\Gamma_{\rm src}\cdot\bar{G}_{\rm bw}\cdot\Gamma_{\rm sink}$.
3. Trace with $G_{\rm fw}^{\rm phased}$ and spatial sum.

The final index structure is `(sink_key, src_key, t)`, and after gathering across MPI ranks the time axis is rolled so that the source time t_0 maps to $t=0$.

5 qTMDWF-like Diagnostic Check

The qTMDWF diagnostic generalizes the two-point function by shifting the backward antiquark line transversely and longitudinally before contracting. For transverse displacement b_T along axis $\hat{n}_T$ and longitudinal displacement b_z:

$$C_{\rm TMDWF}(b_T,b_z,t) \;=\; \sum_{\mathbf{x}} \mathrm{Tr}\left[\Gamma_{\rm src}\,\bar{G}_{\rm bw}^{\rm shift}(\mathbf{x},t;t_0)\,G_{\rm fw}^{\rm phased}(\mathbf{x},t;t_0)\right], \tag{8}$$

where $\bar{G}_{\rm bw}^{\rm shift}$ is the gamma5-conjugated backward propagator shifted by (b_T,b_z). The forward propagator is left unshifted.

This diagnostic is valuable because:

1. It tests that the backward propagator carries the expected momentum and spatial structure.
2. It previews the transverse displacement needed for the full soft factor.

3. For $b_T = 0,\ b_z = 0$ it reduces to a simpler trace that can be compared against the two-point diagnostic as a consistency check.

In the code, `contract_tmdwf_check` iterates over b_T and b_z, applies coordinate-gauge shifts via `prop.shift()`, gamma5-conjugates the shifted propagator, and contracts against the phased forward propagator with a single Γ_{src} insertion (no sink interpolator).

6 Soft-Factor Four-Point Contraction

This is the main measurement. For each source time t_0 and sink separation t_{sep} the contraction uses four Coulomb-gauge wall propagators, organized by their role in the forward/backward quark and antiquark lines:

$$\begin{aligned} G_w &\equiv G_{+\mathbf{k}_{\text{fw}}}(t_0) && \text{(forward quark, source time)} \\ G^{\dagger}_{w,\mathrm{b}_\perp} &\equiv G_{-\mathbf{k}_{\text{bw}}}(t_0) && \text{(backward antiquark, source time)} \\ G^{\dagger}_{w} &\equiv G_{-\mathbf{k}_{\text{fw}}}(t_0 + t_{\text{sep}}) && \text{(forward antiquark, sink time)} \\ G^{\mathrm{b}_\perp}_{w} &\equiv G_{+\mathbf{k}_{\text{bw}}}(t_0 + t_{\text{sep}}) && \text{(backward quark, sink time)} \end{aligned} \tag{9}$$

The naming convention follows the legacy code:

- `Gw`: **G**auge-fixed **w**all propagator for the forward quark at the source.
- `Gw_bperp_dagger`: The dagger emphasizes that this propagator will be gamma5-conjugated ("dagger") and that it corresponds to the backward line that will be transversely displaced ("bperp").
- `Gw_dagger`: Forward antiquark at the sink, gamma5-conjugated.
- `Gw_bperp`: Backward quark at the sink, transversely displaced.

6.1 Sink-Side Momentum Phase

The backward quark propagator at the sink, $G^{\mathrm{b}_\perp}_w$, is multiplied by the momentum-transfer phase

$$G^{\mathrm{b}_\perp}_w \ \longrightarrow\ \exp\!\big(-2i\,\mathbf{P}\cdot\mathbf{x}\big)\ G^{\mathrm{b}_\perp}_w, \qquad \mathbf{P} \equiv \mathbf{k}_{\text{fw}} + \mathbf{k}_{\text{bw}} \tag{10}$$

where $\mathbf{P}$ is the total pion momentum. The factor of 2 arises from the kinematic structure of the soft factor, where both the source and sink sides contribute equal and opposite phases to the four-point function. All other propagators carry their momentum through the wall-source construction; only $G^{\mathrm{b}_\perp}_w$ receives this explicit phase because it is the only propagator for which the wall-source momentum ($+\mathbf{k}_{\text{bw}}$) does not match the required total momentum projection.

6.2 Time-Major Lexicographic Layout

Before contraction, all four propagators are rearranged into *time-major* lexicographic order via

```
prop.lexico(False)
```

This yields the index ordering

$$\underbrace{t}_{\text{time}}\ \underbrace{z}_{\text{slowest spatial}}\ \underbrace{y}_{\text{middle spatial}}\ \underbrace{x}_{\text{fastest spatial}}\ \underbrace{j,\,i}_{\text{spin sink, source}}\ \underbrace{b,\,a}_{\text{color sink, source}} \tag{11}$$

or, in the einsum convention used throughout the code,

```
tzyx j i b a
```
(12)

where Latin letters (i, j, k, l) denote Dirac spin indices and (a, b, c) denote color indices. This time-major layout is essential because the final trace over spatial volume and the time projection must respect the lexicographic order for correct MPI halo exchanges.

7 Closed Spin-Color Blocks

For each transverse displacement vector $\mathbf{b}$ (pointing along the axis $\hat{n}_\mathrm{T}$ with magnitude b_T lattice units), we form two closed spin-color blocks:

7.1 Block A_b – Source Side

$$A_b(\mathbf{x},t) \;=\; G_w(\mathbf{x},t) \;\cdot\; \Gamma_\mathrm{src} \;\cdot\; \gamma_5 \;\cdot\; G_w^{\mathrm{b}_\perp\dagger}(\mathbf{x}+\mathbf{b},\,t)^\dagger \cdot\; \gamma_5\,. \tag{13}$$

In words:

1. Start with the forward quark propagator G_w at position $\mathbf{x}$.
2. Insert the source interpolator Γ_src.
3. Gamma5-conjugate the transversely-shifted backward antiquark propagator $G_w^{\mathrm{b}_\perp\dagger}$ (the dagger in the name refers to the fact that this propagator is stored as $G_{-\mathbf{k}_\mathrm{bw}}$ and must be conjugated).
4. The shift by $+\mathbf{b}$ is applied to $G_w^{\mathrm{b}_\perp\dagger}$ before conjugation.

7.2 Block B_b – Sink Side

$$B_b(\mathbf{x},t) \;=\; G_w^{\mathrm{b}_\perp}(\mathbf{x}+\mathbf{b},\,t)\cdot\; \Gamma_\mathrm{sink} \cdot\; \gamma_5 \;\cdot\; G_w^\dagger(\mathbf{x},\,t)^\dagger \cdot\; \gamma_5\,. \tag{14}$$

In words:

1. Start with the transversely-shifted backward quark propagator $G_w^{\mathrm{b}_\perp}$ at position $\mathbf{x}+\mathbf{b}$. This propagator already carries the sink-side phase, Eq. (10).
2. Insert the sink interpolator Γ_sink.
3. Gamma5-conjugate the forward antiquark propagator $G_w^\dagger$ at position $\mathbf{x}$.

Both blocks A_b and B_b are matrices in spin-color space at each spacetime point $(\mathbf{x},t)$. Their open indices are the spin and color indices that will be contracted in the final trace.

7.3 Four-Point Correlator

With the blocks defined, the soft-factor four-point function is assembled as

$$C_4(t;t_\mathrm{sep},\mathbf{b},\Gamma_1,\Gamma_2) \;=\; \sum_{\mathbf{x}} \mathrm{Tr}\left[A_b(\mathbf{x},t)\;\Gamma_2\;B_b(\mathbf{x},t)\;\Gamma_1\right]. \tag{15}$$

The trace is over the spin and color indices that connect A_b and B_b; the spatial sum $\sum_\mathbf{x}$ projects onto zero (or total) spatial momentum after the phase factors have been accounted for in the propagator definitions.

8 Gamma and Interpolator Conventions

8.1 Soft-Factor Gamma Set

The soft factor uses a reduced set of gamma structures compared to standard meson spectroscopy. The PyQUDA gamma matrices follow the Euclidean conventions of the underlying library:

Key	Matrix	Description
`"5"`	$\gamma_5 = \gamma_5$	Chirality
`"I"`	$\mathbf{1}$	Identity
`"X"`	γ_1	γ_1
`"Y"`	γ_2	γ_2
`"X5"`	$\gamma_1\gamma_5$	$\gamma_1\gamma_5$
`"Y5"`	$\gamma_2\gamma_5$	$\gamma_2\gamma_5$

In the code these are stored in the dictionary `soft_factor_pyquda_gammas`, which maps each key to the corresponding PyQUDA `gamma` object. The `gamma(15)` denotes γ_5 (the 16th element in the standard Euclidean gamma basis, 0-indexed), and `gamma(0)` is the identity.

8.2 Pion Interpolators

The default pion interpolator set is

$$\texttt{"Z5-X5"} \;=\; \gamma_3\gamma_5 - \gamma_1\gamma_5 \;=\; \gamma_3\gamma_5 - \gamma_1\gamma_5\,. \tag{16}$$

This is a linear combination that projects onto the pseudoscalar pion with good overlap at finite momentum. Both Γ_1/Γ_2 (the soft-factor gamma insertions) and `pion_src`/`pion_sink` (the pion interpolators) are passed as Python dictionaries, so the user can supply arbitrary operator sets.

The legacy convention is that the source and sink interpolator dictionaries are the same object by default, but they can be set independently to explore operator dependence.

8.3 Gamma5 as γ_5

The matrix γ_5 appears in three distinct roles:

1. **Gamma5-hermiticity conjugation** of antiquark propagators, Eq. (6).
2. **Block closure** in A_b and B_b, Eqs. (13)–(14).
3. **As a gamma insertion**: when Γ_1 or Γ_2 is `"5"`, it acts as a genuine operator insertion probing the chiral structure of the soft factor.

In the code, the variable `G5 = gamma.gamma(15)` is used for roles 1 and 2, while role 3 uses the entry `soft_factor_pyquda_gammas["5"]`, which is the same underlying matrix.

9 Transverse Displacement Implementation

9.1 Coordinate-Gauge Shifts

The transverse displacement **b** is implemented by a simple array roll in the coordinate gauge (CG). No explicit gauge link is inserted between the shifted quark fields. This is consistent with the CG convention used in the pion qTMD workflow: in Coulomb gauge the gluon field is minimized, and at leading order the gauge link reduces to unity.

For a displacement of b_T lattice units along axis $\hat{n}_T$ (where $n_T = 0$ means x, $n_T = 1$ means y, $n_T = 2$ means z), the operation is

```
xp.roll(Gw_bperp, shift=bT, axis=axis)
```

where `axis = 3 - bT_dir` converts from the physics axis convention to the time-major lexicographic index convention. In the time-major layout `tzyx...`, the spatial axes are ordered ($z = 1, y = 2, x = 3$), so:

`bT_dir`	`axis` in lexicographic layout
0 (x)	3
1 (y)	2
2 (z)	1

9.2 Displacement Parameters

- `bT_dir`: list of axes along which to displace, e.g. `[0]` for x only, `[0, 1]` for both x and y.
- `bT_length`: maximum displacement in lattice units (inclusive; loop runs from 0 to `bT_length`).
- `bz_length`: maximum longitudinal displacement (default 0, used only in the qTMDWF diagnostic).

10 Code Implementation

The core logic lives in the class `pion_soft_factor` (file `pion_soft_factor_vibe_develop.py`). This section documents each method.

10.1 `__init__(parameters)`

Accepts a dictionary with keys:

Key	Description
`quark_mom`	List of quark 3-momenta $[\mathbf{k}_1, \mathbf{k}_2, \dots]$
`bT_dir`	List of transverse displacement axes
`bT_length`	Maximum b_T
`bz_length`	Maximum b_z (qTMDWF diagnostic)
`tsep_list`	List of sink separations
`pion_src`	Dictionary of source interpolators (default: `Z5-X5`)
`pion_sink`	Dictionary of sink interpolators (default: same as src)
`Gamma1`	Dictionary of Γ_1 insertions (default: soft-factor set)
`Gamma2`	Dictionary of Γ_2 insertions (default: soft-factor set)

10.2 `create_wall_src(latt_info, tslice, momentum)`

Constructs the Coulomb-gauge wall source, Eq. (2). Internally it calls `phase.MomentumPhase` to build the momentum phase factor $\exp(+i\mathbf{k}\cdot\mathbf{x})$ and `source.propagator` to place it on the specified time slice.

10.3 `create_wall_propagator(dirac, latt_info, tslice, momentum)`

Assembles the wall source and inverts the Dirac operator: `core.invertPropagator(dirac, wall_src, 1, 0)`. The arguments `1, 0` specify the multigrid level and the number of deflation vectors, respectively (standard single-level inverter with no deflation).

10.4 `save_wall_propagator(prop, tag, attrs=None)`

Saves the propagator to HDF5 using `prop.saveH5(tag + ".h5", "propagator")`. Optional metadata (lattice tag, configuration number, smearing, source time, quark momentum) is written as HDF5 attributes for reproducibility.

10.5 `load_wall_propagator(latt_info, tag)`

Loads a previously saved propagator via `core.LatticePropagator.loadH5(tag + ".h5", "propagator")`.

10.6 `apply_phase(prop, momentum, sign=1, x0=None)`

Multiplies the propagator data by a momentum phase factor:

- Computes the phase $\exp(\pm i\mathbf{k}\cdot(\mathbf{x}-\mathbf{x}_0))$ using `MomentumPhase.getPhase`.
- `sign=1` gives the positive phase; `sign=-1` gives the complex conjugate.
- The phase array is broadcast over the spin-color trailing dimensions.
- Returns a copy of the propagator with the phased data.

10.7 `contract_wall_2pt(latt_info, prop_fw, prop_bw, pion_mom, src_key, sink_keys)`

Computes the wall-to-wall two-point function, Eq. (7). Steps:

1. Phase the forward propagator with $\exp(-i\mathbf{p}\cdot\mathbf{x})$.
2. Lexicographically reorder both propagators: `.lexico(False)`.
3. Gamma5-conjugate the backward propagator.
4. Contract with source and sink interpolators.
5. Trace and spatial sum.
6. Gather across MPI ranks.

10.8 `contract_tmdwf_check(latt_info, prop_fw, prop_bw, pion_mom, src_key)`

Computes the qTMDWF-like diagnostic, Eq. (8). Iterates over (b_T, b_z) and returns a flat list of time-series correlators.

10.9 `contract_soft_factor(latt_info, prop_fw, prop_bw_src, prop_sink_bw, prop_sink_fw, pion_mom)`

The main four-point contraction, described in full in Sec. 11. Returns the five-dimensional array `(n_src, n_gm1, n_dir, n_bT+1, Lt)` together with the ordered key lists.

11 Einsum Contraction Patterns

The soft-factor contraction uses four `einsum` calls per (b_T, Γ_1) combination. This is the most complex contraction in the Pyquda_Measurement codebase. We document the index conventions here.

11.1 Index Convention

In the time-major lexicographic layout, propagators carry the indices

$$\underbrace{t}_{\text{time}}\ \underbrace{z}_{\text{spatial 2}}\ \underbrace{y}_{\text{spatial 1}}\ \underbrace{x}_{\text{spatial 0}}\ \underbrace{j}_{\text{spin sink}}\ \underbrace{i}_{\text{spin source}}\ \underbrace{b}_{\text{color sink}}\ \underbrace{a}_{\text{color source}} \tag{17}$$

Gamma matrices are 4×4 with indices (k, l) in spin space. The interpolator stacks Γ_{src} and Γ_{sink} carry an additional leading batch index s (enumerating the source or sink keys). Similarly, the Γ_1 and Γ_2 stacks carry a batch index over the gamma keys.

11.2 Einsum 1: Closed Block A_b

$$\begin{aligned} A_b \ = \texttt{einsum}\big(&\texttt{tzyxjiba, sik, kl, tzyxmlca, mn} \to \texttt{stzyxjnbc}, \\ &G_w,\ \Gamma_{\text{src}}^{\text{stack}},\ \gamma_5,\ \big[G_w^{\text{b}\perp\dagger}\big]^{\text{shift},\dagger},\ \gamma_5\big) \end{aligned} \tag{18}$$

Index mapping:

Index	Carried by
t, z, y, x	Spacetime (summed/kept depending on position)
j, i	Spin sink/source of G_w
b, a	Color sink/source of G_w
s	Batch over source interpolator keys
k, l, m, n	Spin contraction indices (closed in the einsum)
l, c	Spin/color of $G_w^{\mathrm{b}_\perp\dagger}$ (sink side)
n, c	Output spin/color indices

The superscript "shift, †" means that $G_w^{\mathrm{b}_\perp\dagger}$ has been rolled by b_T along the appropriate spatial axis and then complex conjugated (the dagger in the name). The two γ_5 insertions perform the gamma5-hermiticity conjugation of the shifted propagator.

The output block A_b has shape $(s, t, z, y, x, j, n, b, c)$, where s indexes the source interpolator, (j, n) are the open spin indices, and (b, c) are the open color indices that will connect to B_b.

11.3 Einsum 2: Closed Block B_b

$$\begin{aligned} B_b \;= \texttt{einsum}\big(&\texttt{tzyxjiba, sik, kl, tzyxmlca, mn} \to \texttt{stzyxjnbc}, \\ &G_w^{\mathrm{b}_\perp,\,\mathrm{shift}},\ \Gamma_{\mathrm{sink}}^{\mathrm{stack}},\ \gamma_5,\ \left[G_w^\dagger\right]^\dagger,\ \gamma_5\big) \end{aligned} \tag{19}$$

This is structurally identical to A_b but with different propagators and interpolators:

- $G_w^{\mathrm{b}_\perp,\,\mathrm{shift}}$ is the transversely shifted *and sink-phased* backward quark propagator.
- $G_w^\dagger$ is the forward antiquark propagator from the sink time slice, complex conjugated but *not* shifted.
- Γ_{sink} replaces Γ_{src}.

11.4 Einsum 3: Four-Point Trace

$$\begin{aligned} C_4(\mathbf{x}, t) \;= \texttt{einsum}\big(&\texttt{tzyxjiba, ik, tzyxklba, lj} \to \texttt{tzyx}, \\ &A_b[\mathrm{isrc}],\ \Gamma_2[\mathrm{igm}],\ B_b[\mathrm{isrc}],\ \Gamma_1[\mathrm{igm}]\big) \end{aligned} \tag{20}$$

This einsum closes the spin and color indices:

- j (spin of A_b) $\to$ i (row of Γ_2)
- k (column of Γ_2) $\to$ k (spin of B_b)
- l (spin of B_b) $\to$ l (row of Γ_1)
- j (column of Γ_1) $\to$ j (spin of A_b)
- b, a color indices are traced within `einsum`.

The result is a scalar on spin-color space at each spacetime point $(\mathbf{x}, t)$.

11.5 Einsum 4: Time Projection

$$C_4(t) \;=\; \texttt{einsum}\big(\texttt{tzyx} \to \texttt{t},\ C_4(\mathbf{x}, t)\big) \tag{21}$$

This sums over the spatial volume $\mathbf{x} = (z, y, x)$ at each time slice, yielding the time-series correlator $C_4(t)$. After this step the result is gathered across MPI ranks.

11.6 Full Loop Structure

The contraction loops are nested as follows (outermost first):

1. b_T direction (`idir`)
2. b_T magnitude (0 to `bT_length`)
3. Source interpolator key (`isrc`)
4. Gamma-1 key (`igm`)

For each (b_T, dir) pair, the shifted propagators and the two blocks A_b, B_b are computed *outside* the (isrc, igm) loops. Only the final trace (Einsum 3) and time projection (Einsum 4) are inside the inner loops. This is a significant optimization because the block construction dominates the computational cost.

12 HDF5 Output Structure

12.1 Propagator Files

```
data/pion_soft_factor_prop/
  {lat}.pion_soft_factor_prop.{cfg}.{ama}.{src}.{sm}.{mom}.h5
```

Each file contains a single dataset `"propagator"` and HDF5 attributes: `lat_tag`, `config_num`, `sm_tag`, `tslice`, `quark_momentum` (as int32 array).

12.2 Two-Point Diagnostic

```
data/pion_soft_factor_c2pt/
  {lat}.pion_soft_factor_c2pt.{cfg}.{ama}.{src}.{sm}.fw_{mom1}.bw_{mom2}.h5
```

HDF5 group hierarchy (from `save_pion_soft_factor_c2pt_hdf5_noRoll`):

```
/SS/{src_key}/{sink_key}/PX{p[0]}PY{p[1]}PZ{p[2]}   -> dataset
```

12.3 qTMDWF Diagnostic

```
data/pion_soft_factor_qTMDWF/
  {lat}.pion_soft_factor_qTMDWF.{cfg}.{ama}.{src}.{sm}.fw_{mom1}.bw_{mom2}.h5
```

HDF5 group hierarchy (from `save_pion_soft_factor_qTMDWF_hdf5_noRoll`):

```
/SP/{src_key}/PX{px}PY{py}PZ{pz}/{b_dir}/bT{bT}/bz{bz}   -> dataset
```

12.4 Four-Point Soft Factor

```
data/pion_soft_factor/
  {lat}.pion_soft_factor.{cfg}.{ama}.{src}.{sm}.fw_{mom1}.bw_{mom2}.h5
```

HDF5 group hierarchy (from `save_pion_soft_factor_hdf5_noRoll`):

```
/src{src_key}_sink{sink_key}/{gm1}_{gm2}/{b_dir}_{bT}/ts{tsep}   -> dataset
```

Each dataset contains a 1D complex array of length L_t, the time-series correlator for that combination of source interpolator, gamma insertions, transverse displacement, and sink separation.

13 Comparison with Legacy Code

This implementation is a port from the GPT/PyQUDA mixed workflow `PyQUDA_qTMD_ff_4pt_einsum.py`. The key differences are:

1. **Pure PyQUDA**: No GPT dependency. All gauge fixing, smearing, inversion, and I/O use the PyQUDA stack.

2. **HDF5 propagator storage**: Propagators are saved between Stage 1 and Stage 2, enabling (a) decoupled production and analysis, (b) reuse of propagators for multiple contraction parameter sets, and (c) portability across machines (propagators can be analyzed locally).
3. **Identical physics**: The contraction patterns, gamma conventions, phase factors, and wall-source definitions are preserved from the legacy workflow to ensure output compatibility. The numerical results should agree to machine precision with the GPT/PyQUDA mixed code.
4. **Class-based design**: The `pion_soft_factor` class encapsulates all parameters and provides methods for each computational step, improving maintainability and enabling unit testing.
5. **Backend abstraction**: All `einsum` calls are dispatched through `xp` (which can be `numpy`, `cupy`, `dpnp`, or `torch`), so the same code runs on CPU, NVIDIA GPU, Intel GPU, or PyTorch backends without modification.

Acknowledgments

This work builds on the PyQUDA framework and the legacy GPT/PyQUDA mixed soft-factor code. The implementation follows the conventions established in the pion qTMD and qTMDWF analyses.

References

[1] J. C. Collins and D. E. Soper, "Back-To-Back Jets in QCD," Nucl. Phys. B **193**, 381 (1981).

[2] J. C. Collins and D. E. Soper, "The Theorems of Perturbative QCD," Ann. Rev. Nucl. Part. Sci. **37**, 383 (1987).

[3] X. Ji, J. Ma and F. Yuan, "QCD factorization for semi-inclusive deep-inelastic scattering at low transverse momentum," Phys. Rev. D **71**, 034005 (2005) [arXiv:hep-ph/0404183].

[4] X. Ji, Y. Liu and Y.-S. Liu, "Transverse-momentum-dependent parton distribution functions from large-momentum effective theory," Phys. Lett. B **811**, 135946 (2020) [arXiv:1911.03840 [hep-ph]].